# Automated Construction of the User Interface for a CERIF-Compliant Research Management System

Gordana Milosavljević[a], Dragan Ivanović[a], Dušan Surla[b], Branko Milosavljević[a]
[a]Faculty of Technical Sciences, Novi Sad
[b]Faculty of Sciences, Novi Sad

## Abstract

**Purpose –** The goal is to construct a user interface for CERIF-compatible research management systems in an automated way. The system has to support all data items defined by the CERIF data model. The result of the automated construction is a functioning software system that can be customised according to the specific needs by manual changes in the program code.

**Design/methodology/approach –** The Model-Driven Architecture (MDA) approach to software engineering is based on the idea of automated construction of software applications based on the model of a system. System modelling usually utilises the Unified Modelling Language (UML). A specific UML profile (extension) used for describing user interface-related aspects of the system model is defined. It is used to define the behaviour of the user interface in the model itself. By relying on the concept of a standard generic screen form implementing common data management operations and tools for automatic code generation it is possible to obtain a fully functional software application that is completely defined by the system model.

**Findings –** A proposal of an automated user interface construction method for CERIF-compatible research management systems is defined. The proposed application enables the management of all CERIF-specified data items and facilitates the customisation of the model according to the specific needs. The customisation process is automated by using the tools for user interface code generation based on human-computer interaction (HCI) standard defining the layout and behaviour of generic screen forms. The amount of automatically generated application functionality justifies the use of the model-driven approach to software development.

**Research limitations/implications –** The method shown enables automated construction of a software application providing data management and standard reporting. However, this approach is not suitable for generating specific complex reports such as author bibliographies because the structure of such reports can not be easily standardised as is the case with screen forms.

**Practical implications –** Using the model of a system it is possible to efficiently produce a functional software application that can further be customised according to the needs of specific institutions and users. The customisation process itself is made efficient since it is iteratively performed using the same cycles of modelling and software generation. The proposed method for automated user interface construction is used for implementing the web application for research management at the University of Novi Sad.

**Originality/value –** A proposal for efficient software system development and its customisation is given. This way a software application is obtained that can be quickly adapted to new requirements and used in different research management systems based on a CERIF-compatible data model.

**Keywords** CERIF, Model-Driven Architecture, code generation tools

## 1 Introduction

Research management systems play a significant role in the development of science (Zimmerman, 2002). Standardisation of a data model used in those systems is necessary to enable exchanging data between those systems and to enable scientists and researchers to find the desired data in various research management systems. CERIF (Common European Research Information Format) (Asserson et al., 2002) is a standard that describes data model enabling interoperability between research management systems. CERIF describes a physical data model (Jörg et al., 2009a) and an XML data exchange format (Jörg et al., 2009b). It is often extended in order to support the features of particular research management systems.

This paper describes the generic application structure, the development method, and tools used for automated user interface construction of a research management system based on CERIF. The presented solution can use different mechanisms for data storage (relational databases, XML native databases, etc.) according to the needs of particular implementations. The presented implementation uses the CERIF extension presented in (Ivanović et al., 2010) for data storage. The user interface construction tools can be used for creating web applications, as well as classical standalone GUI applications. Special care is taken about the support for multilingual data items (e.g., paper titles and abstracts).

Generic application structure was developed in order to avoid redundancy in the program code. All application elements dynamically adapt their layout and behaviour according to the invocation context. Rather than implementing an application element for every situation that occurs during a software product lifetime, we strive to develop a generic application composed of adaptable components that can be customised for particular requirements with the minimal number of lines of code written.

Our development method and tools are based on Model-Driven Architecture (MDA) principles (Kleppe et al, 2003). MDA is based on the idea that the knowledge about a system being observed and the functions of the software being constructed can be represented using appropriate models at different levels of abstraction. Models at the higher abstraction levels independent of a specific technology platform (Platform Independent Model – PIM) are used for automatic transformations to models at the lower abstraction levels dependent of a specific platform (Platform Specific Model – PSM). Platform specific models are used for automatic code generation, or for direct run-time model execution within an appropriate virtual machine.

In order to overcome the problems of integration and exchange of information among different tools, UML was used to model all aspects of an application. However, UML without extensions is not suitable for modelling the user interface (da Silva and Patton, 2003; van den Bergh and Coninx, 2005; Paterno, 2001), so we has developed a UML profile [9] that enables the specialisation of UML for modelling the user interface of information systems applications. The specialisation using UML profiles consists of: (1) defining stereotypes that represent extensions to existing UML metaclasses, (2) defining tags that represent stereotype attributes, and (3) defining constraints that may pose additional restrictions on existing UML constraints [9].

Using custom UML profiles for modelling complex systems has a number of advantages. A majority of UML-based general-purpose modelling tools may be used, so the domain-specific language and the supporting tools can be quickly created. Profile and model exchange among different modelling tools is facilitated by the XMI mechanism [9]. One model may combine multiple profiles [9], so an entity from the observed system can be described from different aspects without the need to create multiple independent models and transformation rules. UML is a rich modelling language and all its resources may be used in model development (not just the subset related to the domain-specific language).

The rest of the paper is structured as follows. Section 2 reviews the related work. Section 3 presents the model-driven development method used in this paper. Section 4 reviews the concepts of generic application components. Section 5 describes code generation tools that provide automated software construction based on the previously developed models. One part of a model of a CERIF-compliant system and part of a UML profile for user interface modelling are described in Section 6. Section 7 presents the results obtained by automated generation of the user interface for the presented model. The last section concludes the paper and outlines some directions of future work.

## 2 Related Work

**CERIF extensions.** The CERIF data model is often extended in order to support the features of particular research management systems. The papers (Asserson et al., 2002, Jeffery et al., 2000) describe a data model extension that was created in order to satisfy requirements of a research management system that was developed at the University of Bergen. Furthermore, the paper (Asserson et al., 2002) describes a data model extension that was created in order to satisfy requirements of research management system that was developed on "Science and Technology Facilities Council". A data model extension that was created in order to satisfy requirements of the IST World portal is described in the papers (Jörg et al., 2005; Kiryakov et al., 2005; Ferlež, 2005; Jörg et al., 2006). The papers (Jeffery et al., 2000; Jeffery, 2000) describe an extension that uses the formalized Dublin Core [1] for description of published results of scientific research. The paper (Ivanović et al., 2010a) describes CERIF-compatible data model based on MARC 21 format [2]. Extending CERIF for evaluation of scientific research results is presented in (Ivanović et al., 2010b).

**Relations between research and library management systems.** There are many research management systems available today: IST World [3], HunCRIS [4], Frida [5] etc. In recent years, the relation between research management and librarian systems has been discussed (Joint, 2008). The paper (Dijk et al., 2006) describes the NARCIS portal representing an access point to all scientific research information in the Netherlands. That system is an integration of the Netherlands research management system and DARENET (Digital Academic Repositories in the Netherlands). Furthermore, the paper (Olivier, 2009) describes collaboration between research management system and digital library at Pretoria University. Also, there are similar collaborations between a research management system and a digital library at Glasgow University, Southampton University and Kingston University (Olivier, 2009). Imperial College Digital Repository [6] is a digital repository for research output of Imperial College London. That system is described in the paper (Afshari and Jones, 2007). The main part of a research management system is management of data about published scientific research results. It is reasonable that data about publications are stored in some bibliographic format because bibliographic formats are rich in metadata and because of easier collaboration and integration of research management systems with library information systems. The papers (Belić and Surla, 2008a; Belić and Surla, 2008b) describe a model and an implementation of a system for entering data about published results in UNIMARC format [7].

**The BISIS system.** The results presented in this paper were obtained within the development of the BISIS library information system [8]. The BISIS system has been developing since 1993 at the University of Novi Sad. The current version 4 is based on XML technology. Within that version, an XML editor for cataloguing in the UNIMARC and MARC 21 format (Dimić and Surla, 2009; Dimić et al, 2010) that uses the component for generating catalogue cards (Rađenović et al, 2009) is developed. The papers (Boberić and Surla, 2009, Milosavljević et al, 2010) describe an XML editor for searching and downloading bibliographic records in accordance with Z39.50 standard, the papers (Tešendić et al, 2009; Milosavljević and Tešendić, 2010) describe a library circulation system, and the paper (Vidaković et al., 2009) deals with distributed library catalogues in BISIS. The subject matter of the paper (Rudić and Surla, 2009) is the application of XML for conversion of bibliographic records in MARC 21 format. The conversion is implemented by XSLT technology.

**User interface development methods.** Different aspects of user interface development are the subject of a number of papers. However, the majority of presented methods is hardly applicable to the development of real systems because (1) too much time and effort is needed to implement and synchronise different user interface models, (2) the knowledge about the system cannot easily be shared among models, (2) the lack of appropriate development tools, and (4) the lack of consensus on which models should describe the user interface (Seffah and Gaffar, 2007). It is hard to find examples of user interfaces generated based on declarative models that really operate in a real environment (da Silva, 2000).

Another problem is that the majority of user interface modelling tools use their own notation, thus complicating the integration with models of other parts of an application (da Silva, 2000; da Silva and Patton, 2003). This leads to the problem of integrating the user interface with other parts of the system that implement business logic.

The code generation tools and the corresponding UML profile represent the authors' research results in the domain of software engineering (Milosavljević et al., 2003a; Milosavljević et al., 2003b; Milosavljević and Perišić, 2003; Milosavljević and Perišić, 2004; Dejanović et al., 2010).

## 3 The method of system development

Developing software according to the MDA is performed in phases, where the system model takes the central role. The starting model that is used to specify the system elements and behaviour is created in a way that it does not depend on a particular implementation platform. It is named the *platform independent model* (PIM). The goal of this stage is to preserve the knowledge about the observed system regardless to later changes in the development platform. A PIM consists of a number of UML diagrams describing the static and/or dynamic aspects of the system. Upon creating the PIM, MDA tools using transformation rules will produce one or more *platform-dependent models* (PSM) and *bridges* among them. The transformation itself is an automatic generation of the target model based on the starting model. The transformation definition is a set of rules describing how concepts of the source language map to concepts of the target language. The aim of creating PSMs is to enable developers to add platform-specific information to the model. That information may be used to increase the percentage of the program code that is automatically generated. PSMs, supplied with that information, are used for automatic code generation or for direct run-time model execution within an appropriate virtual machine. Figure 1 presents our development artefacts in the context of PIM and PSMs.

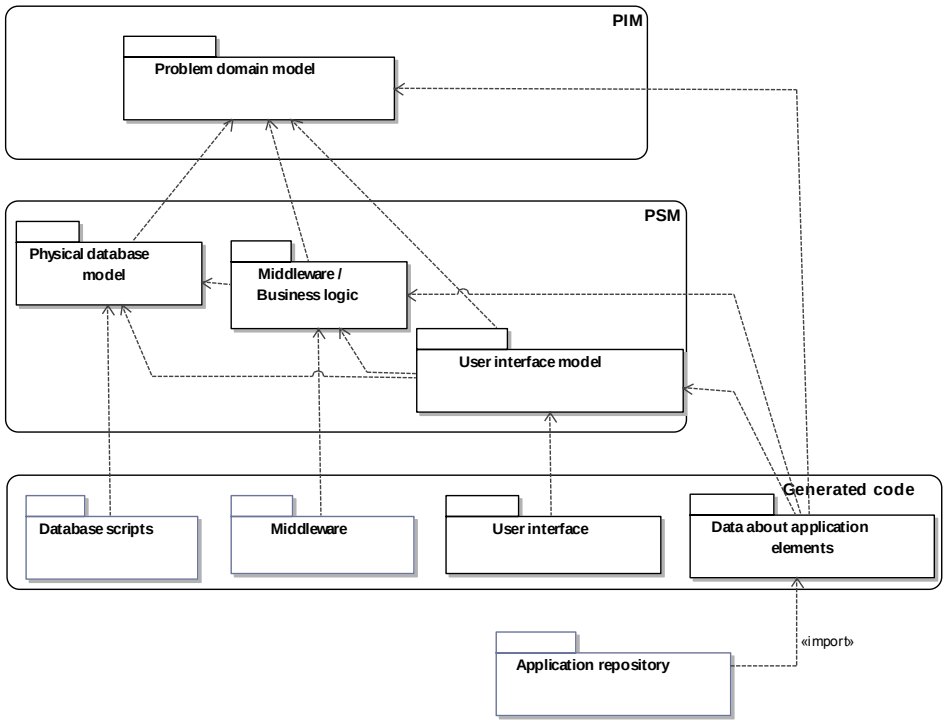



Figure 1 The models of a system and their transformations

The development of PIM (Figure 1) requires domain knowledge and the knowledge of the modelling language. Since PIM does not contain details regarding software development, its development may be carried out by both system developers and end users. A PIM is primarily composed of class diagrams. Other UML artefacts may be used in modelling and documenting different aspects of the business domain, but they do not take part in automatic generation of PSMs.

A PSM consists of a database model, a model of business logic operations, and a user interface model. The transformation to database model and middleware model is supported by many modelling tools (for example, see [10, 11]). The transformation to the user interface model has to be carried out according to the rules of the tools being used. The goal of this transformation is to obtain as large percentage of the user interface-related generated code as possible.

When a PSM is supplied with elements not automatically obtained from PIM, artefacts representing the system implementation may be generated.

- Scripts for database schema creation/change are generated from database model.
- The implementation of middle-tier components according to the chosen platform (Java, .NET) is generated from the business logic model.
- A fully functional user interface is generated from the user interface model. The implementation is based on a chosen technology (web applications, Java Swing, .NET WinForms, etc).
- Atomic transactions implementing common data management operations are generated from the database model.
- All PSM models contribute to application metadata stored in the application repository. This metadata represent a basis for application administration, defining user environments, user roles and groups.

## 4 Generic GUI components

In order to obtain quality program code and shorten the time needed to implement the software product, it is necessary to avoid redundancy in the program code. One way of achieving this goal is to use a "normalised" application architecture that is based on a normalised database schema. In most cases, one system entity has a single application artefact – a standardised screen form, report, etc. All application elements need to dynamically adapt their layout and behaviour according to the invocation context. Rather than implementing an application element for every situation that occurs during a software product lifetime, we strive to develop a generic application composed of adaptable components that can be customised for particular requirements with the minimal number of lines of code written (ideally, without changes to the program code at all). This way, the development team implements application components as "raw material", and specific application subsystems and user roles are defined (administered) as different views on a generic application.

The basis of this application architecture is our human-computer interaction (HCI) standard aimed at defining functional and visual features of application components. Its goals include the following: simplicity of use, quick user training, and the automation of user interface construction. From the users' standpoint, the existence of a HCI standard is needed to achieve consistent layout and behaviour across the whole application, thus shortening training time, improving productivity and reducing errors during application use. From the developers' standpoint, the HCI standard prevents improvisation, facilitates team work and project management, and facilitates the maintenance of complex software systems [12].

The development of the HCI standard defining the layout and behaviour of generic screen forms aimed at finding the minimal set of functions needed to support business activities. In cases when generic screen forms cannot satisfy user requirements, the standard may be extended with new types of screen forms or a particular form may be manually customised to meet the requirements. The paper (Milosavljević et al., 2004b) defines a number of types of screen forms. For this discussion, the following types are relevant:

- standard data management form,
- standard panel,
- parent-child form, and
- many-to-many form.

**Standard form** is designed to display data and all available operations so the user can choose a data item and invoke an operation on it without memorising commands (the object-action approach, (Shneiderman, 1998)). Standard operations common to all entities are represented by buttons/icons at the top of the form, while specific operations (if they exist) are represented by links/buttons at the right hand side. The standard form layout is presented in Figure 2, while examples of its implementation are presented in Figures 9 and 13.

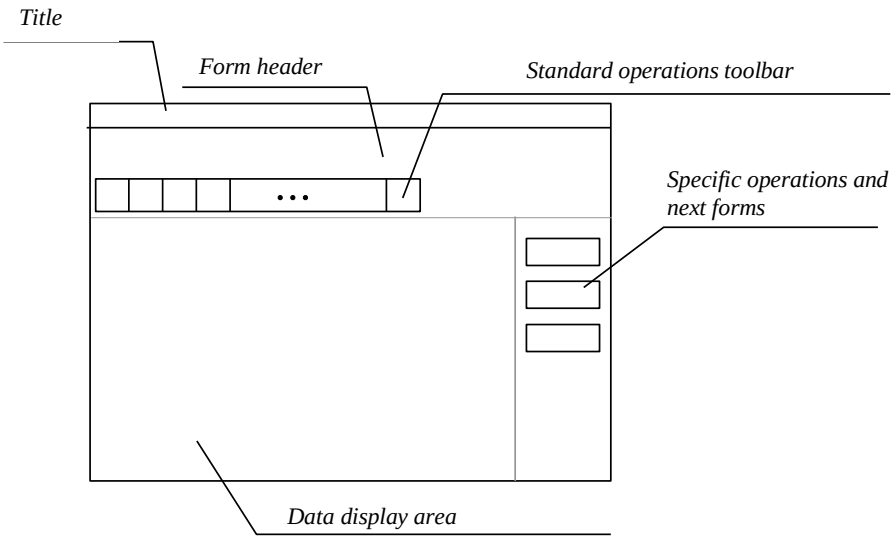


Figure 2 The layout of the standard screen form

Operations common to all entities include search (query by form), display, addition, update, removal, and copying. Specific operations include complex data processing procedures associated with the given entity (transactions), invocation of related (*next*) screen forms, and invocation of reports. The standard mandates that the specific operations always use the currently selected (viewed) record.

**Standard panel** has the appearance and the behaviour of the standard form but instead being shown in its own window, it is used as an element of a complex form. Standard panels are regularly used for parent-child and many-to-many forms.

Standard forms are designed to minimise the use of menus. When a user is situated in the context of the current business task, he or she can access all additional information and operations related to that context directly at the current position within the application, without the need to go back to the menu. Instead of a menu-driven application having a strict hierarchical structure and a fixed way of using it, this application is based on functionally comprehensive screen forms that facilitate users' navigation and the choice of the most appropriate path. The screen form acts as a graph node that enables traversal towards all connected nodes that make sense in the current context. This way, the application adopts the structure of a hypermedia document, enabling the implementation in both desktop and web application platforms.

Relationships among screen forms are represented by three mechanisms: *zoom*, *next*, and *activate*. The **zoom** mechanism represents the invocation of the form associated with the given entity where the user can choose a data item and "drag" it (pick its values) to the fields of the previously viewed form. A *combo zoom* represents a combo box filled with the data of the entity where a user can choose a desired value. If the value does not exist, a form associated with that entity can be invoked and add the value needed. A *zoom* is invoked by clicking the button with ellipsis ("..."). An example of a *zoom* mechanism for the selection of the language used for the journal title is presented in Figure 10.

The **next** mechanism, invoked from the form associated with the current entity, displays the form associated with the child entity with its data filtered so that only connected objects are displayed. The key or a representation of the parent entity is displayed in the form header, so the user easily recognises the current context. A *next* can be invoked by menu items, buttons, or links. An example of a group of *next* buttons is presented in Figure 9 (the *Connections* frame).

The **activate** mechanism enables direct invocation of a form by another form, without restrictions on the data displayed. The invoked form does not need to be related to the current one.

A **parent-child form** is used for data that have hierarchical structure, where each element in the hierarchy is modelled as an entity in the database. In this case, the *next* mechanism does not possess sufficient functionality since the user doesn't want to separate related data in multiple forms – a single form displaying multiple levels in data hierarchy is needed. Each element in the hierarchy is represented by a standard panel, where a panel at the *n*-th hierarchy level filters its content according to the selected data item at the level *n-1*.

Since fetching data from related child entities affects application performance, parent-child forms should be used only when necessary, while the *next* mechanism should be used in other cases. The layout of the parent-child form is presented in Figure 3. Examples of implementation of a parent-child form are displayed in Figures 10 and 11.

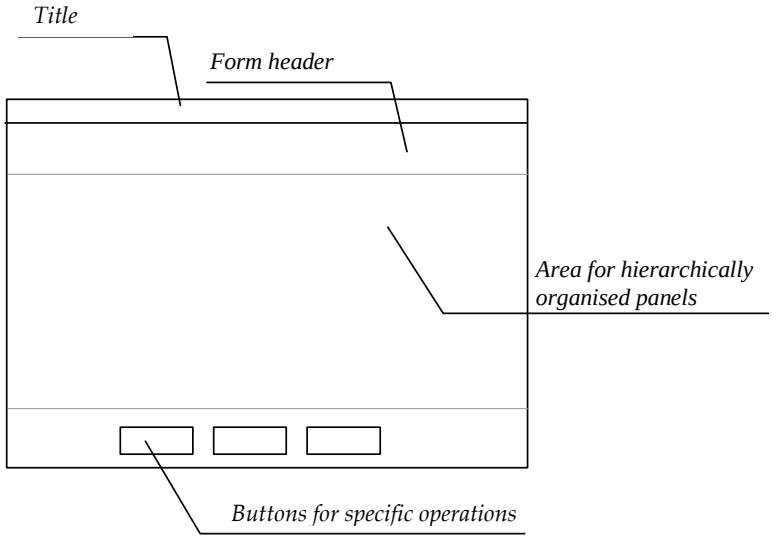



Figure 3 The layout of the parent-child screen form

The **many-to-many form** is used for intensive management of data belonging to entities connected by "many-to-many" relationships, with or without associate classes. Its layout is presented in Figure 4, and its example in Figure 12. This screen form is used as follows:

- A number of desired records are selected in the upper panel. These records are "dragged" to the lower panel by clicking the button with the downwards arrow. If a record is dragged by mistake, it can be revoked back by clicking the upwards arrow button.
- The values of non-key attributes of a record selected in the lower panel may be changed.

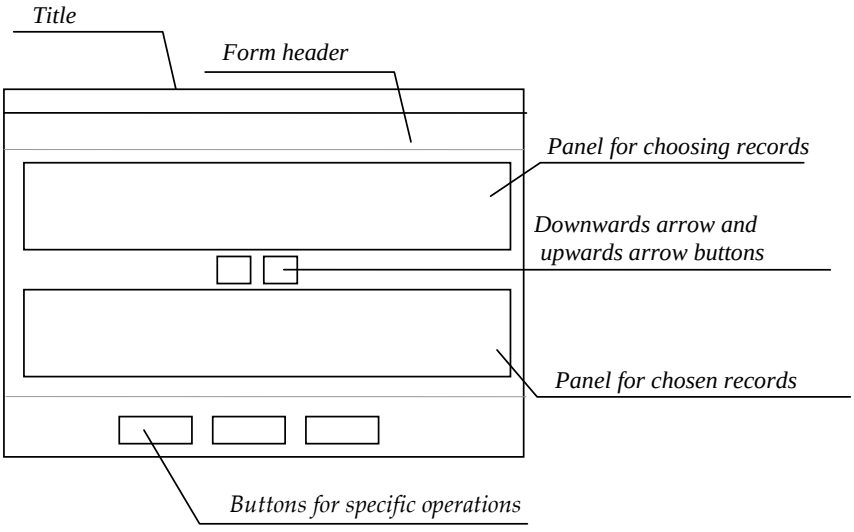



Figure 4 The layout of the many-to-many screen form

## 5 System development tools

Tools for efficient software development should support the presented development method (Section 3) and user interface concepts (Section 4). These tools should automate as much repetitive operations as possible

and provide for reuse of artefacts in different phases of the project and system documentation as a direct product of development. UML modelling tools that supports user-defined languages and transformations, object/relational mapping, and model synchronisation are available today. For examples of such modelling tools see [10, 11]. In order to obtain a fully functional application based on a system model, we need an additional set of tools. This section reviews such tools presented in details in papers (Milosavljević et al., 2003a; Milosavljević et al., 2003b; Milosavljević and Perišić, 2003; Milosavljević and Perišić, 2004). A high-level view of the tool architecture is presented in Figure 5.

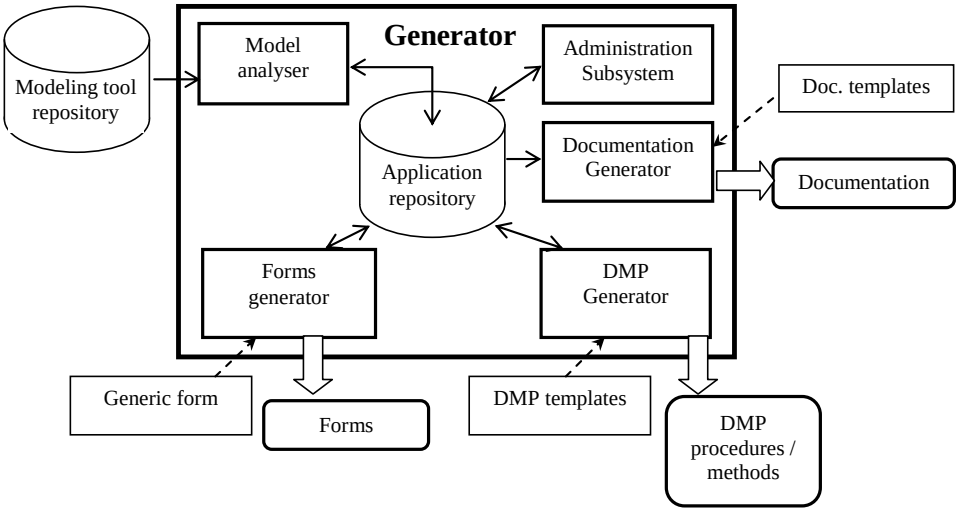



Figure 5 The architecture of the code generation tools

*Model Analyser* is used for fetching models from the modelling tool's repository, storing those data in the application repository, and generating the initial propositions of the application components.

*Forms generator* is a GUI application featuring viewing/changing the initially proposed application components and data describing the application. The manual changes applied using this tool are preserved in the case of a repeated cycle of analysis and generation.

*Data manipulation procedure (DMP) Generator* provides (1) automatic generation of atomic data transactions for addition, changing, and removal for all database entities and (2) efficient interactive development of complex transactions based on a number of data management patterns.

*Administration Subsystem* is used for user and group management, access control administration, definition of subsystems and associating them to users and groups.

*Documentation Generator* provides (1) repository-based documentation on implemented application elements and (2) analysis/visualisation tool for procedure invocation trees.

## 6 The model of a CERIF-compliant system

The development of a CERIF-compliant research management system starts with the creation of the PIM. Although the CERIF specification (Jörg et al., 2009a) defines a detailed data model that may be used as a basis for an information system, we have not adopted it directly for the following reasons. Firstly, we strive to model a user interface that is designed around a natural grouping of data from the users' standpoint. This model can further be used in different CERIF-compliant systems provided there is a transformation from the derived to the original CERIF model. An example of a customised CERIF-compliant model is given in (Ivanović et al., 2010). Next, automated generation of a fully functional software application needs a model with greater expressiveness so that the tools can generate the application based on the additional information contained in the model.

This paper presents only a part of the comprehensive CERIF-compliant model that deals with managing data about papers published in scientific journals. The results presented here can be applied to the whole model. Figure 6 presents a PIM that models published journal papers.

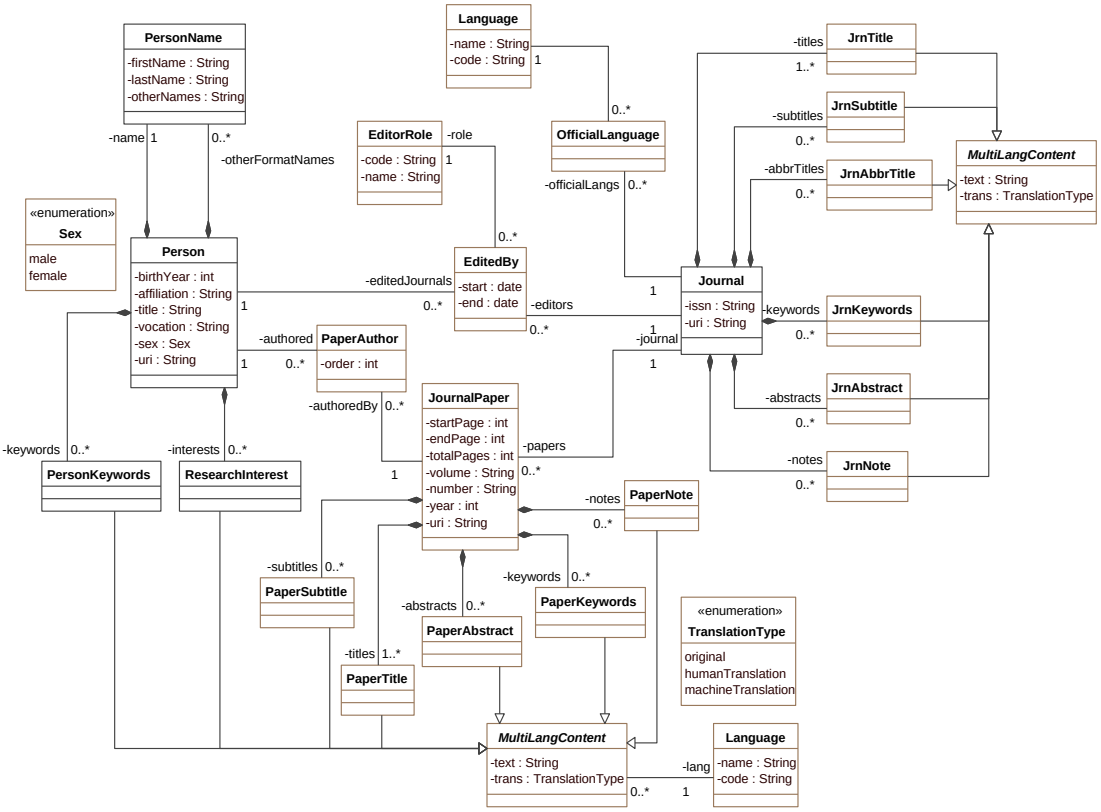



Figure 6 A platform-independent model of a part of CERIF-compliant system

A scientific journal is represented by the *Journal* class. Data describing the journal include its ISSN number and the journal's Internet address (URI), as well as contents that may be defined in multiple languages: journal title (*JrnTitle*), subtitle (*JrnSubtitle*), abbreviated title (*JrnAbbrTitle*), keywords (*JrnKeywords*), abstract (*JrnAbstract*), and notes (*JrnNote*).

All multilingual contents in this model are described by three attributes: textual content, language designation, and the origin of the text (original text, human translation, or machine translation). The structure of multilingual content is represented by the abstract class *MultiLangContent* that is inherited by all classes representing actual multilingual contents. The *Language* class represents a language, while *TranslationType* determines the origin of the text.

A paper published in a journal is represented by *JournalPaper*. A *JournalPaper* is associated to the *Journal* in which it is published, while other elements of paper description are attributes (*volume*, *number*, *year*, *startPage*, *endPage*, *totalPages*, and *uri*). Multilingual textual contents included in paper description are represented by separate classes connected to *JournalPaper* by composition. These are *PaperTitle*, *PaperSubtitle*, *PaperAbstract*, *PaperKeywords*, and *PaperNote*.

Persons, possibly having different roles, are represented by *Person*. A *Person* has a *name* of *PersonName* type. A *Person* may have many variants of the main name, all of which are represented by *otherFormatNames*. The other data describing a *Person* are the class' attributes (*birthYear*, *affiliation*, *title*, *vocation*, *sex*, *uri*, *keywords*, and *interests*). In a complete CERIF-compliant model, *affiliation* would be represented by another class. For the sake of brevity, we have modelled it here as a simple string.

The fact that one *Person* may be the author of many *JournalPapers*, while a *JournalPaper* may be a work of many *Persons*, is represented by the *PaperAuthor* class. Similarly, *EditedBy* models the editor role of a *Person* in a *Journal*.

Based on the previously described PIM, a PSM model is generated. The PSM is supplied additional information about the behaviour of the user interface that is based on the presented HCI standard. This additional information is expressed by using stereotypes (defined in the UML profile) that represents particular HCI

concepts. The PSM is further supplied with information needed for a detailed specification of the application. The PSM representing the user interface is presented in Figure 7.

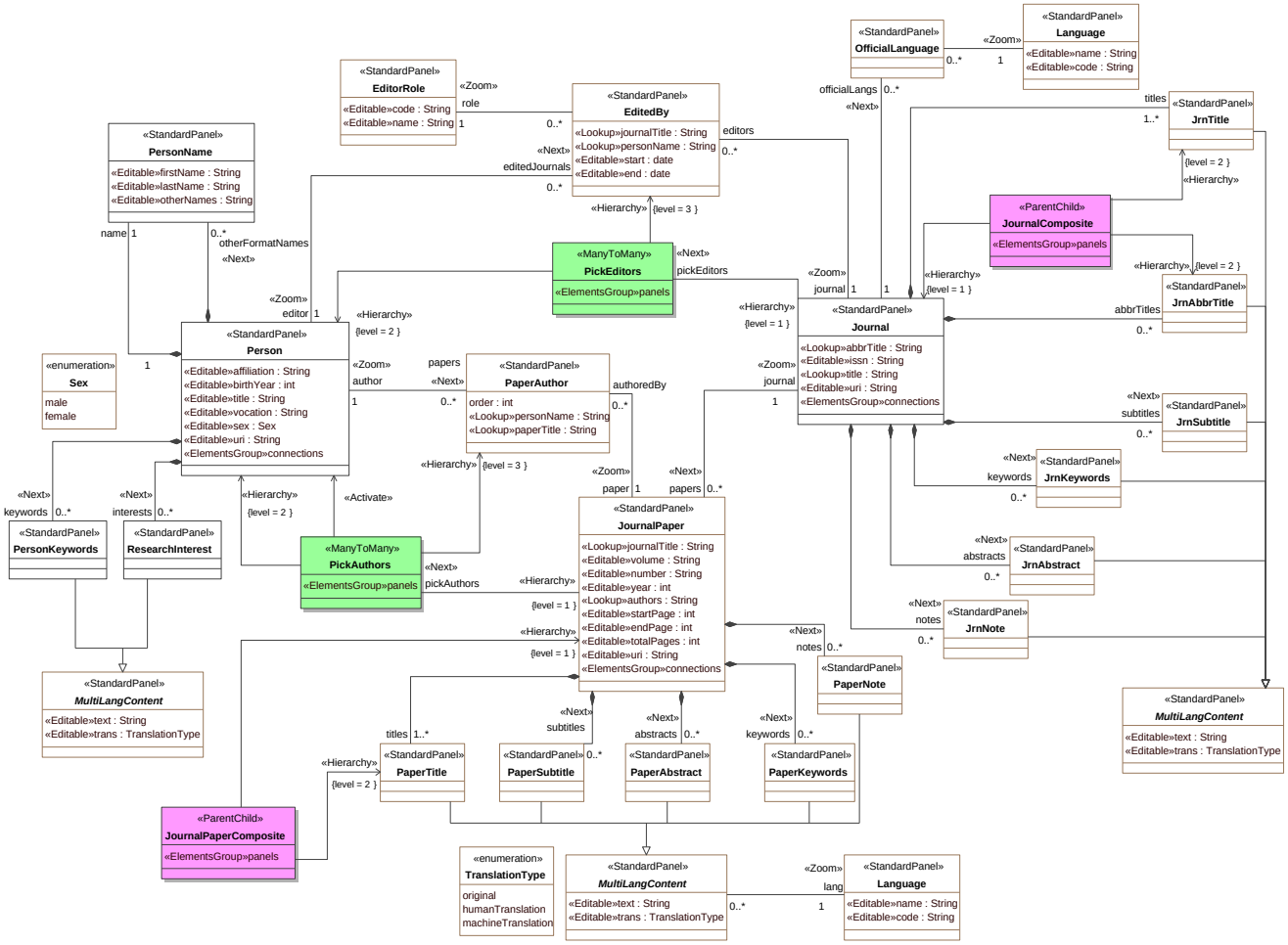



Figure 7 A platform-specific model of the user interface

Figure 8 presents a part of the UML profile that is used for modelling the PSM presented in Figure 7. The complete profile consists of 37 stereotypes and 9 enumerations.

All stereotypes inherit the *VisibleElement* stereotype (see Figure 8) that extends the *Element* metaclass. It represents an element of the model that has a corresponding element in the user interface. It can be used to specify the type of GUI component (from the *ComponentType* enumeration), a label and visibility. Since *Element* metaclass is an abstract ancestor of all UML metaclasses, we can use this to represent any element in the PSM that has a GUI representation and a label. The enumeration *ComponentType* comprises components independent of the implementation platform.

*VisibleClass* stereotype represents a class that is mapped to a panel in the application GUI. If the panel is associated with an empty window (or a web page), it becomes a screen form that may be independently invoked. A panel may be used standalone or as a component in a complex panel (descendants of *ContainerPanel* stereotype). "Visible classes" used in the model in Figure 7 are *ParentChild*, *ManyToMany*, and *StandardPanel*.

A class with the *ParentChild* stereotype models a parent-child screen form. The hierarchy must have at least two levels. The maximum number of levels is limited by the ease of use and the available screen size.

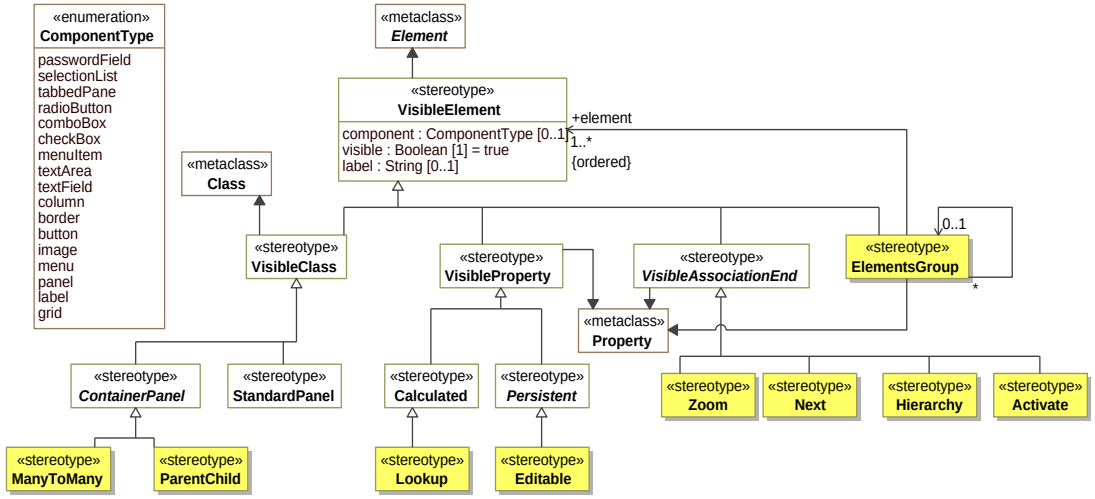



Figure 8 A part of the UML profile used in the model

The position of a panel in the hierarchy is defined by hierarchical association between it and the container panel. Hierarchical association is an association whose end attached to the contained panel has the *Hierarchy* stereotype (discussed later). Panels in the hierarchy may be simple panels or other parent-child panels. Classes stereotyped with *ParentChild* in the PSM (see Figure 7) are *JournalPaperComposite*, specifying the layout of the form for managing journal papers (see the generated form in Figure 11), and *JournalComposite* for managing journals (see Figure 10).

A class stereotyped with *ManyToMany* models a many-to-many form. The role of contained panels is defined by the hierarchical association between a contained panel and the composite panel. Classes stereotyped with *ManyToMany* in the PSM are *PickAuthor*, specifying the layout of the form for choosing journal paper authors (see the generated form in Figure 12), and *PickEditors* for choosing journal editors.

The *StandardPanel* stereotype gives a persistent class the visual representation as a standard panel. The *StandardPanel* tags provide the selection of allowed operations (add, update, delete, search), default mode when the panel is displayed (add, update, or search), data filter and sorting options. An example of tag values for a *StandardPanel* is given in Table 1.

Table 1 An example of the tag values for a *StandardPanel*

| Tag name | Value |
|---|---|
| component | ComponentType::panel |
| label | Person |
| visible | true |
| add | true |
| update | true |
| delete | true |
| search | true |
| defaultOperationMode | OperationMode::updateMode |
| confirmDelete | true |
| dataFilter | |
| sortBy | name.lastName, name.firstName |

An example of the standard panel layout for *Person* is presented in Figure 9. The same panel embedded in *PickAuthors* with context-specific settings applied is presented in Figure 12.

The context-specific settings are contained in association ends (descendants of the *VisibleAsocciationEnd* stereotype) connecting the activated panel with the embedded panel. These settings provide for redefinition of the attributes of the standard panel as well as the change in its layout and behaviour – by setting a list of panel elements that should not be visible or enabled in the given context. The

*VisibleAssociationEnd* descendants introduce additional tags according to their purpose. Our PSM uses *Zoom*, *Next*, and *Activate* descendants that represent the HCI concepts of zoom, next, and activate mechanisms, respectively. Also, *Hierarchy* is used to define the role of an embedded panel in a composite (*ParentChild* or *ManyToMany*) panel.

Tag values for a *Hierarchy* stereotype are illustrated by the association between *PickAuthors* and *Person* (see Table 2). If a tag value is not set, and there is a tag with the same name in the panel, its value is used as default (for example, *label*, *component*, *dataFilter*, and *sortBy*).

Table 2 Tag values for *Hierarchy* stereotype for the association end from *PickAuthors* to *Person*

| Tag name | Value |
|---|---|
| label | |
| component | |
| visible | true |
| hiddenProperty | birthYear, title, vocation, sex, url |
| disabledProperty | |
| hiddenOperation | |
| hiddenGroup | connections |
| add | false |
| update | false |
| delete | false |
| search | true |
| defaultOperationMode | OperationMode::searchMode |
| dataFilter | |
| sortBy | |
| level | 2 |

A specific tag for *Hierarchy* is *level*, which meaning depends on the type of the panel used. For a *ManyToMany* panel, *level* 1 denotes the form header, *level* 2 denotes the panel for choosing records, and *level* 3 is for the panel containing the selected records. When used with a *ParentChild* panel, level 1 is the tree root while *level n* (where $n>1$) denotes child panels of the panel with *level n*-1. The list of all *level* tag values in *Hierarchy* association ends that are used for a *ParentChild* panel must not have gaps.

The *Zoom* stereotype introduces only one additional tag, *comboZoom*, denoting that the given zoom should be represented by a combo box. The values in the combo box are fetched from the attribute that represents the whole entity (e.g., journal title, paper title, person's first name + last name).

The Next stereotype introduces the following tags: *autoActivate* (indicates that the *next* form should be invoked automatically upon entering the new record in this panel), *displayIdentifier* (indicates that the *next* form should present an identifier of the current record in its header), and *displayRepresentative* (indicates that the *next* form should include the representative attribute of the parent record in its header).

Attributes of classes having a visual representation are assigned *VisibleProperty* descendants. *VisibleProperty* defines tags that customise the display format, label text, default values, etc. *VisibleProperty* descendants used in the PSM are *Editable* and *Lookup*.

The *Editable* stereotype denotes a persistent visible attribute editable by the user. An example is the attribute *volume* in the class *JournalPaper*. Table 3 lists tag values for this attribute.

Table 3 Tag values for *Editable* stereotype of the *volume* attribute in the class *JournalPaper*

| Tag name | Value |
|---|---|
| component | ComponentType::textField |
| label | Volume |
| columnLabel | Volume |
| visible | true |

| | |
|---|---|
| representative | false |
| disabled | false |

The *Lookup* stereotype represents a visible property whose value is calculated using the values of the referenced record. The lookup expression can be formulated (1) using an OCL expression or (2) by stating the class whose representative attribute values are fetched. An example of *Lookup* tags (see Table 4) is given for the *journalTitle* attribute of *JournalPaper*.

Table 4 Tag values for *Lookup* stereotype of the *journalTitle* attribute in the class *JournalPaper*

| Tag name | Value |
|---|---|
| component | ComponentType::textField |
| label | Title |
| columnLabel | Title |
| visible | true |
| displayFormat | |
| disabled | true |
| lookupClass | |
| expression | journal.titles.text \| journal.titles.trans = TranslationType::original |

The *ElementsGroup* stereotype is used to group elements (attributes or methods) in a class, thus forming semantic segments represented by groups of GUI components. The *label* tag represents a name for the group (frame title, panel name, or tab name). A group is composed of its elements, has an orientation (horizontal or vertical), the position within the panel (header, data area, area for specific operations), visual alignment, and group owner. An example in the PSM is the *panels* group in the *JournalComposite* class (see Table 5).

Table 5 Tag values for *ElementsGroup* stereotype for the *panels* group in the class *JournalComposite*

| Tag name | Value |
|---|---|
| label | Details |
| component | ComponentType::border |
| visible | true |
| element | journal, jrnTitle, jrnAbbrTitle |
| orientation | GroupOrientation::vertical |
| location | GroupLocation::componentPanel |
| alignment | GroupAlignment::left |
| groupOwner | |

## 7 Generating the user interface

The model presented in Figure 7, along with the tools presented in Section 5, can be used for automated generation of the user interface based on concepts presented in Section 4. The result is a web application based on the Java platform that provides data management according to the presented model.

Figure 9 presents a screen form for managing data about persons that is composed of two frames. At the top of the *Details* frame there is a grid display of persons already registered in the database. Clicking the *Add*, *Delete*, and *Search* buttons invokes standard operations. The fields below the grid represent a detailed view into the selected record. These fields provide for changing the data as well. The *Connections* frame contains buttons that invoke *next* forms related to the selected person.

Figure 10 presents a ParentChild form for managing data about journals. As in the previous example, the *Details* frame contains a grid display of the journals already registered and fields for a detailed view of the selected journal. Associations of a Journal with other classes are represented in two ways. Journal title and abbreviated title, being the most important for the user, are displayed directly in this for by means of embedded

(child) panels named *Titles* and *Abbreviated Titles*. Connections to other related objects (subtitles, abstracts, keywords, etc.) are presented as *next* buttons in the *Connections* frame.

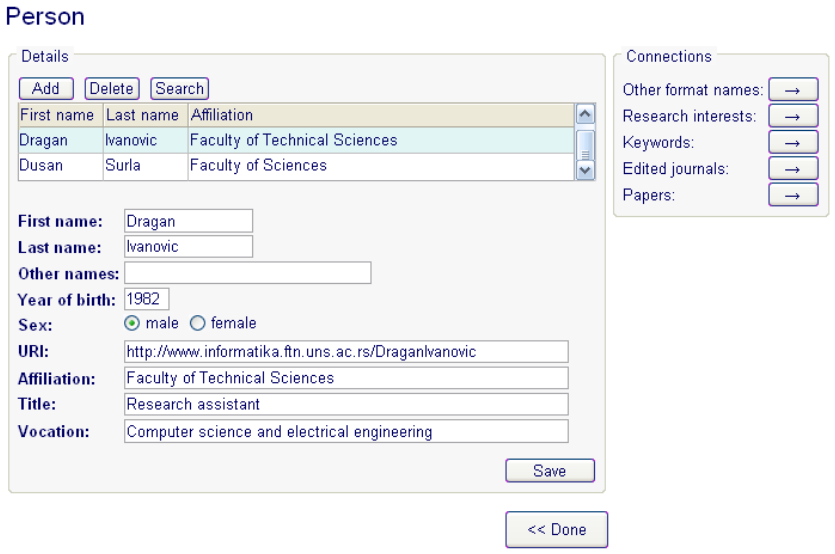



Figure 9 Managing data about persons

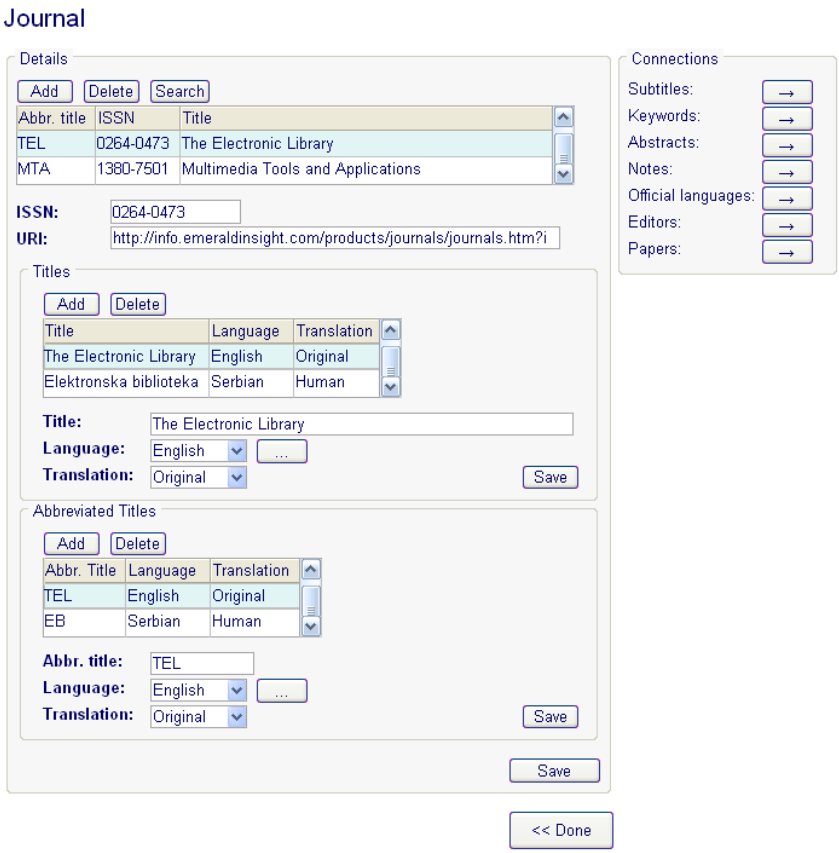



Figure 10 Managing data about journals

Managing data about journal papers utilizes the screen form presented in Figure 11. This is another implementation of a parent-child form. The *Details* frame contains a grid displaying the papers already registered, fields presenting a detailed view into the selected paper, and an embedded (child) panel for displaying paper title(s). The related *next* forms are invoked using the buttons in the *Connections* frame.

The screen form for connecting authors to papers is an implementation of a many-to-many form. The upper part presents a list of all registered persons. It is necessary to find a desired author in this list and move it to the list of authors currently associated with the given paper (the list in the lower part). If an author is not currently registered, clicking the *Add author* button (the use of the *activate* mechanism) will invoke a standard form for managing persons presented in Figure 9. Upon registering the new person, the user is transferred back to this form for picking paper authors, this time with the desired author present in the list.

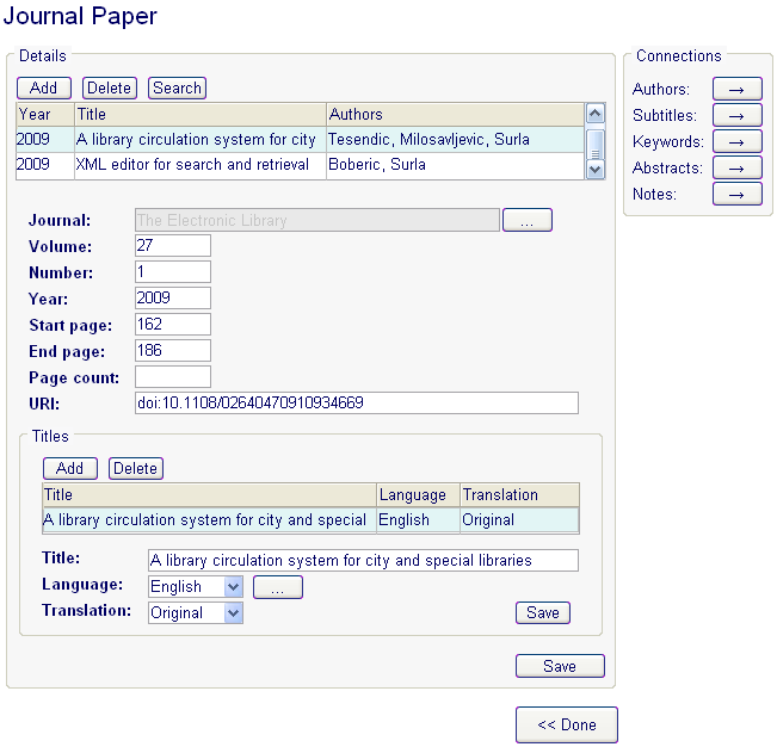



Figure 11 Managing data about journal papers

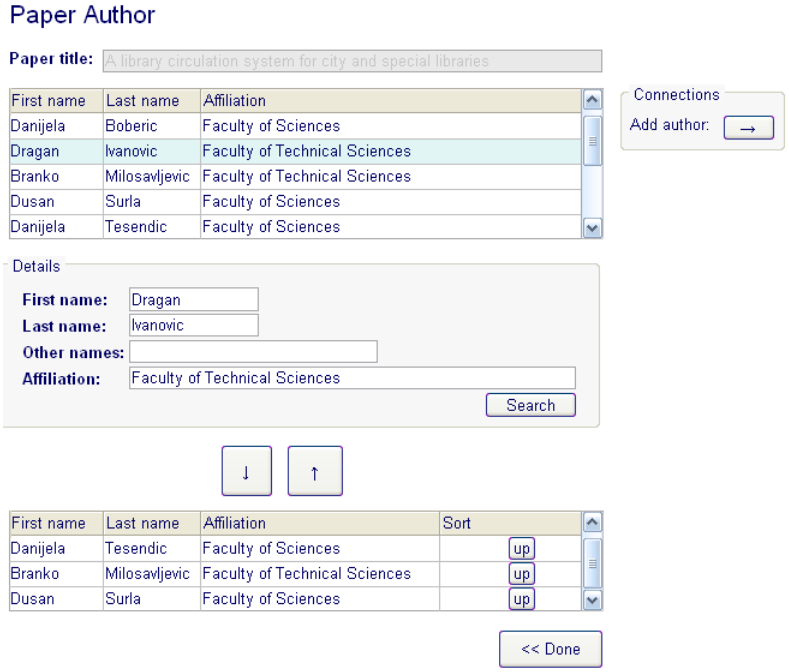



Figure 12 Connecting authors to the journal paper

Figure 13 presents a standard for managing paper abstracts. This form is invoked by clicking the appropriate *next* button in the form for managing papers (see Figure 11). This is an example of a standard form usually invoked in a predefined context – in this case, it is the paper whose abstracts are being entered. Hence, the tabular view of abstracts includes only the abstracts associated with the given paper previously selected in the form in Figure 11.

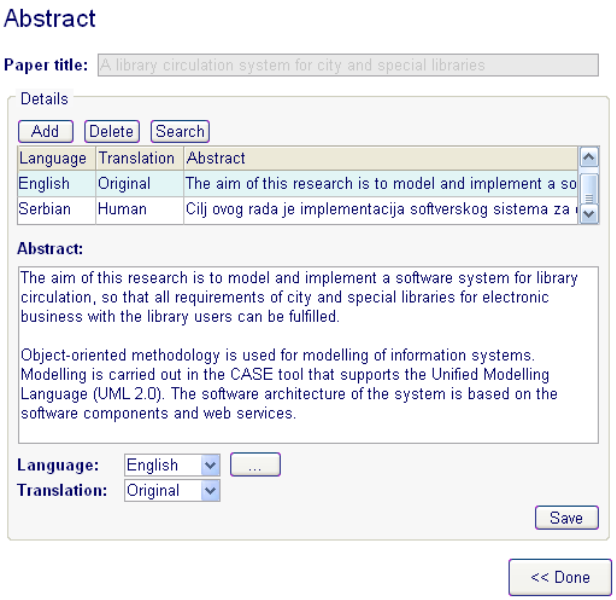



Figure 13 Managing abstracts for a journal paper

## 8 Conclusions

The results presented in this paper confirm that a user interface for a CERIF-compliant research management system can be constructed in an automated way. The resulting system supports all elements specified by CERIF and has a user interface that is consistent in layout and behaviour across the whole application. The resulting system can be further adapted to specific requirements in two ways:

1. New requirements may be met by changes in the system model. These requirements are then implemented in the same way as the original requirements – by another modelling and code generation cycle. This enables rapid, incremental, iterative, prototype-based development of a software application that represents a research management system.
2. The result of code generation is a fully functional software application. Specific requirements that cannot be represented in the model or are not easily represented by the application architecture concepts may be implemented by manual changes to the program code.

The presented development method does not depend on a particular implementation platform. The same system development concepts can be applied to different implementation platforms.

This paper has presented a part of a data model defined by CERIF. Further application of the presented method includes the development of a complete CERIF-compliant model, supplied with additional information needed for generating the user interface and other implementation elements.

Further research within the development of the presented system includes the standardisation of generic database reports similarly to the presented standardised screen forms.

### References

Afshari, F. and Jones, R. (2007), "Developing an integrated institutional repository at Imperial College London" *Program: Electronic library and information systems*, Vol. 41, No.4, pp. 338-352.

Asserson, A., Jeffery, K., Lopatenko, A. (2002), "CERIF: Past, Present and Future: An Overview", *Proceedings of the 6th International Conference on Current Research Information Systems, University of Kassel, August 29 - 31, 2002*, pp. 34-42.

Belić, K. and Surla, D. (2008a), "Model of a User Friendly System for Library Cataloguing", *ComSIS*, Vol. 5 No. 1, pp. 61-85.

Belić, K. and Surla, D. (2008b), "User Friendly Web Application for Bibliographic Material Processing", *The Electronic Library*, Vol. 26, No. 3, pp. 400-410.

Boberić, D. and Surla, D. (2009), "XML Editor for Search and Retrieval of Bibliographic Records in the Z39.50 Standard", *The Electronic Library*, Vol. 27, No 3, pp. 474-95.

Dejanović, I., Milosavljević, G., Tumbas, M., Perišić, B. (2010). "A Domain-Specific Language for Static Structure Definition of Database Oriented Applications", *Computer Science and Information Systems*, Vol. 7, No. 2 (in print).

Dijk, E., Baars, Chr., Hogenaar, A. and Meel, M. van (2006), "NARCIS: The Gateway to Dutch Scientific Information", *Elpub 2006 Conference, Bansko, June 14-16, 2006*, pp. 49-57.

Dimić, B. and Surla, D. (2009), "XML Editor for UNIMARC and MARC21 cataloguing", *The Electronic Library*, Vol. 27, No 3, pp. 509-528.

Dimić, B., Milosavljević, B., and Surla, D. (2010), "XML schema for UNIMARC and MARC 21 formats", *The Electronic Library*, Vol. 28, No. 2, pp. 245-262.

Ferlež, J. (2005), "Public IST World Deliverable 1.3 – Data Models for Representation of Expertise", 12 p., available at: http://ist-world.dfki.de/downloads/deliverables/ISTWorld_D1.3_DataModelForRepresentationOfExpertise.pdf (accessed 26 April 2010)

Ivanović, D., Surla, D., and Konjović, Z. (2010a), "CERIF-compatible data model based on MARC 21 format", *The Electronic Library*, (in print).

Ivanović, D., Surla, D., and Racković, M. (2010b), "A CERIF data model extension for evaluation and qualitative expression of scientific research results". *Scientometrics*, Vol. 83. (in print), available at: http://www.springerlink.com/content/y766146340438u51/fulltext.pdf (accessed 26 April 2010)

Jeffery, K., Asserson, A. and Revheim, J. (2000), "CRIS, Grey Literature and the Knowledge Society", *Proceedings CRIS-2000*, Helsinki, 22 p., available at: ftp://ftp.cordis.europa.eu/pub/cris2000/docs/jeffery_fulltext.pdf (accessed 26 April 2010)

Jeffery, K. (2000), "An architecture for grey literature in a R&D context", *The International Journal on Grey Literature*, Vol. 1, No. 2, pp. 64-72.

Joint, N. (2008), "Current research information systems, open access repositories and libraries", *Library Review*, Vol. 57, No 8, pp. 570-575.

Jörg, B., Ferlež, J. and Grabczewski, E. (2005), "Public IST World Deliverable 1.2 – Data Model for Knowledge Organisation", 18 p., available at: http://ist-world.dfki.de/downloads/deliverables/ISTWorld_D1.2_DataModelForKnowledgeOrganisation.pdf (accessed 26 April 2010)

Jörg, B., Ferlež, J., Grabcewski, E. and Jermol, M. (2006), "IST World: European RTD Information and Service Portal", *8th International Conference on Current Research Inforation Systems: Enabling Interaction and Quality: Beyond the Hanseatic League (CRIS 2006)*, Bergen, Norway, 10 p., available at: http://epubs.cclrc.ac.uk/bitstream/905/ISTWorld01.pdf (accessed 26 April 2010)

Jörg, B., Jeffery, K., Asserson, A. and van Grootel, G. (2009a) "CERIF 2008 - 1.0 Full Data Model (FDM) Introduction and Specification", available at: http://www.eurocris.org/fileadmin/cerif-2008/CERIF2008_1.0_FDM.pdf (accessed 26 April 2010)

Jörg, B., Krast, O., Jeffery, K. and van Grootel, G. (2009b), "CERIF 2008 – 1.0 XML Data Exchange Format Specification", available at: http://www.eurocris.org/fileadmin/cerif-2008/CERIF2008_1.0_XML.pdf (accessed 26 April 2010)

Kiryakov, A., Grabczewski, E., Ferlež, J., Uszkoreit, H. and Jörg, B. (2005), "Public IST World Deliverable 1.1 – Definition of Central Data Structure", 23 p., available at: http://ist-world.dfki.de/downloads/deliverables/ISTWorld_D1.1_CentralDataStructure.pdf (accessed 26 April 2010)

Kleppe, A., Warmer, J., Bast, W. (2003), "MDA Explained: The Model Driven Architecture: Practice and Promise", Addison-Wesley.

Milosavljević, B., Vidaković, M., Komazec, S., Milosavljević, G. (2003a), "User Interface Code Generation for EJB-Based Data Models Using Intermediate Form Representations", *Principles and Practice of Programming in Java*, Kilkenny, Ireland. pp. 125-128.

Milosavljević, B., Vidaković, M., Komazec, S., Milosavljević, G. (2003b), "User Interface Code Generation for Data-Intensive Applications with EJB-based Data Models", *Software Engineering Reseaarch and Practice (SERP'03)*, Las Vegas, NV, USA. pp. 23-27.

Milosavljević, G., Perišić, B. (2003), "Really Rapid Prototyping of Large-Scale Business Information Systems", *IEEE Intl Workshop on Rapid System Prototyping*, San Diego, CA, USA. pp. 100-106.

Milosavljević, G., Perišić, B. (2004). "A Method and a Tool for Rapid Prototyping of Large-Scale Business Information Systems", *Computer Science and Information Systems*, Vol. 1, No. 2, pp. 57-82.

Milosavljević, B., Boberić, D. and Surla, D. (2010), "Retrieval of Bibliographic Records Using Apache Lucene", *The Electronic Library* (in print).

Milosavljević, B. and Tešendić, D. (2010), "Software Architecture of Distributed Client/Server Library Circulation System", *The Electronic Library*, Vol. 28, No. 2, pp. 286-299.

Olivier, E., (2009), "Open Scholarship and research reporting in tandem: creating more value for the University of Pretoria", *The African Digital Scholarship & Curation Conference, May 12-14, 2009,* 5 p., available at: http://www.ais.up.ac.za/digi/docs/olivier_paper.pdf (accessed 26 April 2010)

Paterno, F. (2001), "Towards a UML for Interactive Systems", Proceedings of *8th IFIP Working Conference on Engineering for Human-Computer Interaction (EHCI'01)*, Toronto, Canada. pp. 7-18.

Rađenović, J., Milosavljević, M. and Surla, D. (2009), "Modelling and implementation of catalogue cards using FreeMarker" *Program - Electronic Libarary and Information Systems*, Vol. 43, No. 1, pp. 63-76.

Rudić, G. and Surla, D. (2009), "Conversion of bibliographic record to MARC 21 format", *The Electronic Library*, Vol. 27, No 6, pp. 950-967.

Seffah, A. and Gaffar A. (2007), "Model-based user interface engineering with design patterns", *Journal of Systems and Software*, Vol. 80, No. 8, pp. 1408-1422.

Shneiderman, B. (1998). *Designing the User Interface: Strategies for Effective Human-Computer Interaction*, 3rd edition, Addison-Wesley.

da Silva, P.P. (2000), "User Interface Declarative Models and Development Environments: A Survey", Proceedings of DSV-IS2000, *Lecture Notes in Computer Science*, Vol. 1946, pp. 207-226.

da Silva, P.P., Patton, N.W. (2003), "Improving UML Support for User Interface Design: A Metric Assessment of UMLi", Workshop on Bridging the Gaps Between Software Engineering and Human-Computer Interaction at *International Conference on Software Engineering* (ICSE '03), Portland, Oregon. pp. 76-83.

Tešendić, D., Milosavljević, B. and Surla, D. (2009), "A Library Circulation System for City and Special Libraries", *The Electronic Library*, Vol. 27, No. 1, pp. 162-168.

Vidaković, M., Milosavljević, B., Konjović, Z., Sladić, G. (2009). "Extensible Java EE-based Agent Framework and Its Application on Distributed Library Catalogues", *Computer Science and Information Systems*, Vol. 6, No. 2, pp. 1-28.

Zimmerman, E. (2002), "CRIS-Cross: Current Research Information Systems at a Crossroads", *Proceedings of the 6th International Conference on Current Research Information Systems*, University of Kassel, August 29 - 31, 2002, pp. 12-21.

**Web sites**

[1] *Dublin Core Metadata Initiative*, accessed 13 October 2009, <http://dublincore.org/>

[2] *MARC 21 Standards (Network Development and MARC Standards Office, Library of Congress)*, accessed 13 October 2009, <http://www.loc.gov/marc/ >

[3] *IST World EU project*, accessed 13 October 2009, <http://ist-world.dfki.de >

[4] *HunCRIS* , accessed 13 October 2009, <http://nkr.info.omikk.bme.hu/HunCRIS_eng.htm>

[5] *FRIDA*, accessed 13 October 2009, <http://www.uio.no/english/research/frida/>

[6] Imperial College Digital Repository, accessed 13 October 2009, <http://spiral.imperial.ac.uk/>

[7] *UNIMARC (IFLANET)*, accessed 13 October 2009, <http://archive.ifla.org/VI/3/p1996-1/sec-uni.htm>

[8] Library software system BISIS, accessed 13 October 2009, <http:// bisis.uns.ac.rs/>

[9] *OMG Unified Modelling Language (OMG UML) Superstructure, V2.1.2*, accessed 10 January 2010, <http://www.omg.org/spec/UML/2.1.2/Superstructure/PDF>

[10] *Sybase PowerDesigner*, accessed 10 January 2010, <http://www.sybase.com/products/modelingmetadata/powerdesigner>

[11] *MagicDraw*, accessed 10 January 2010, <http://www.magicdraw.com>

[12] Schumacher, Jr, R.M.. (1996). *Ameritech Graphical User Interface Standards and Design Guidelines*, accessed 10 January 2010, <http://www.usercentric.com/publications/2009/12/03/ameritech-graphical-user-interface-standards-and-design-guidelines>